

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

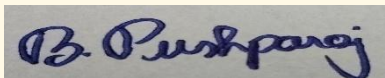
CO3 ASSESSMENT TOOL 1 – Git & Docker Technical Assignment

Course Code:	CSA1008	Course Title:	Software Engineering
CO Assessed:	CO3	Assessment:	Assessment Tool 1 – Git & Docker Technical Assignment
Weightage:	20%	Total Marks:	25
CO3 Statement:	Build full-stack applications with an emphasis on containerization, version control, and collaborative development		
Student Name:	B. Pushparaj	Reg. No.:	192421136
Date:	19.8.2026	Duration:	60 minutes

Code of Conduct

I Pushparaj .B (192421136) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____



Annexure A – Git & Docker Technical Assignment

Instructions to Students:

Each student has been assigned an individual problem statement. Based on the assigned problem statement, prepare a Git & Docker Technical Assignment by applying version control and containerization concepts. Your report should include appropriate diagrams, screenshots, commands, and explanations wherever applicable.

Assignment Questions

1. Problem Analysis

- Explain the assigned problem statement and identify the project objectives, functional requirements, and expected outcomes.

2. Project Structure Design

- Design and explain the folder structure of your project repository.
- Justify the purpose of each major folder and file included in the repository.

2. Git Repository Initialization

- Create a Git repository for the project.
- Explain the steps involved in repository initialization and describe the purpose of the essential Git files.

2. Git Branching Strategy

- Design a suitable branching strategy (e.g., Main, Development, Feature, Release, Hotfix).
- Justify why the selected branching model is appropriate for the assigned project.

2. Version Control Workflow

- Demonstrate the Git workflow by performing meaningful commits, branch creation, merging, conflict resolution (if applicable), and repository synchronization.
- Explain the purpose of each Git operation performed.

2. Commit History Analysis

- Present the commit history of the project.
- Explain how commit messages follow good version control practices and contribute to project maintenance.

2. Docker Environment Design

- Explain why Docker is suitable for the assigned project.
- Identify the application components that require containerization.

2. Dockerfile Development

- Design and explain the Dockerfile required to containerize the application.
- Describe the purpose of each instruction used in the Dockerfile.

2. Docker Compose Design

- Develop a Docker Compose configuration for the project.
- Explain the services, networking, volumes, environment variables, and dependencies used.

2. Container Deployment and Testing

- Demonstrate the execution of the application using Docker containers.

- Explain the testing procedure and provide evidence that the application runs successfully.

2. Benefits of Git and Docker

- Discuss how Git and Docker improve collaboration, version management, portability, deployment, scalability, and maintainability for your assigned project.

2. Challenges and Improvements

- Identify the challenges encountered during implementation.
- Suggest possible improvements and future enhancements for the Git workflow and Docker deployment.

Expected Deliverables

- Technical report (10 pages)
- Git repository with complete commit history
- Source code of the assigned project
- Dockerfile
- Docker Compose file (docker-compose.yml)
- Screenshots of Git operations and Docker execution
- Architecture and workflow diagrams

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Analysis & Project Design(20%)	Clearly explains the problem statement, objectives, requirements, expected outcomes, and well-organized project structure with proper justification.	Good analysis and project structure with minor omissions.	Basic analysis and repository structure with limited justification.	Incomplete or unclear analysis and project organization.

Git Repository & Branching Strategy(20%)	Repository initialized correctly with appropriate branching strategy, clear workflow, and proper repository organization.	Repository and branching strategy are mostly correct.	Basic Git setup with limited branching implementation.	Incorrect or incomplete Git repository and branching strategy.
Version Control Workflow & Commit History(10%)	Demonstrates meaningful commits, branching, merging, synchronization, conflict resolution, and professional commit messages with clear history.	Most Git operations demonstrated correctly with good commit history.	Limited Git workflow and commit practices.	Poor workflow and inadequate commit history.
Docker Implementation(20%)	Well-designed Docker environment, Dockerfile, and Docker Compose configuration with clear explanation of services, networking, volumes, and dependencies.	Functional Docker implementation with minor issues.	Basic Docker implementation with limited explanation.	Incomplete or incorrect Docker implementation.
Deployment, Testing & Results(15%)	Application successfully deployed and tested with appropriate screenshots and evidence	Deployment completed with minor issues.	Partial deployment and testing demonstrated.	Deployment unsuccessful or insufficient evidence.

	of successful execution.			
Documentation, Challenges & Future Enhancements(15%)	Professional report (10–15 pages) with diagrams, screenshots, references, discussion of benefits, challenges, and future improvements.	Good documentation with minor omissions.	Basic documentation with limited discussion.	Poor documentation and insufficient analysis.

Criteria	Mark
System Requirement Analysis (30%)	/5
Selection of Software Architecture (25%)	/5
Architecture Diagram and Component Design (20%)	/5
Component Interaction and Quality Attribute Analysis (15%)	/5
Architectural Justification and Technical Presentation (10%)	/5
TOTAL	/25

GIT & DOCKER TECHNICAL ASSIGNMENT

Topic: Accessible Learning Platform.

1. Problem Analysis

1.1 Problem Statement

Students with visual impairments face difficulties while using conventional online learning platforms because many systems do not provide proper accessibility features. Problems include difficult navigation, inaccessible documents, lack of voice support, and assessments that are not suitable for screen-reader users.

The proposed Accessible Learning Platform provides an inclusive digital learning environment with screen-reader support, voice navigation, audio learning resources, accessible assessments, course management, and progress tracking.

1.2 Project Objectives

- Provide an accessible online learning environment.
- Support students using screen readers.
- Provide voice-based navigation.
- Allow teachers to upload accessible learning materials.
- Provide audio-based learning resources.
- Conduct accessible online assessments.
- Track student learning progress.
- Provide teachers/admins with a simple management dashboard.

1.3 Functional Requirements

1. Student registration and login.
2. Teacher registration and login.
3. Course creation and management.

4. Upload learning materials.
5. Screen-reader-friendly interface.
6. Voice navigation.
7. Audio learning content.
8. Online accessible assessments.
9. Student progress tracking.
10. Admin management.
11. Report generation.

1.4 Expected Outcome

The system provides a user-friendly and accessible learning platform where students with different accessibility needs can learn, attend assessments, and track their progress independently.

2. Project Structure Design

Accessible-Learning-Platform/

```
|
|— app.py
|— requirements.txt
|— Dockerfile
|— docker-compose.yml
|— README.md
|
|— database/
|   |— database.sql
|
|— templates/
|   |— login.html
|   |— register.html
|   |— dashboard.html
|   |— courses.html
|   |— course_detail.html
```

```

|   ├── assessment.html
|   ├── progress.html
|   └── admin_dashboard.html
|
|   ├── static/
|   |   ├── css/
|   |   |   └── style.css
|   |   ├── js/
|   |   |   └── script.js
|   |   └── audio/
|   |
|   └── uploads/
|       ├── documents/
|       └── audio/

```

Purpose of Files

File/Folder	Purpose
app.py	Main Flask application
requirements.txt	Python dependencies
Dockerfile	Creates application container
docker-compose.yml	Manages application and database
database/	Stores database scripts
templates/	HTML pages
static/css/	Website styling
static/js/	JavaScript functionality
static/audio/	Audio learning resources
uploads/	Uploaded learning materials
README.md	Project documentation

3. Git Repository Initialization

Git is used to maintain different versions of the Accessible Learning Platform.

Commands

```
mkdir Accessible-Learning-Platform
```

```
cd Accessible-Learning-Platform
```

```
git init
```

```
git add .
```

```
git commit -m "Initial project setup"
```

```
git branch -M main
```

Connect to GitHub

```
git remote add origin <repository-url>
```

```
git push -u origin main
```

Important Git Files

`.git/`

The `.git` folder stores Git's version-control information.

`.gitignore`

It prevents unnecessary files such as Python cache files, virtual environments, and sensitive configuration files from being uploaded.

Example:

`venv/`

`__pycache__/`

`*.pyc`

`.env`

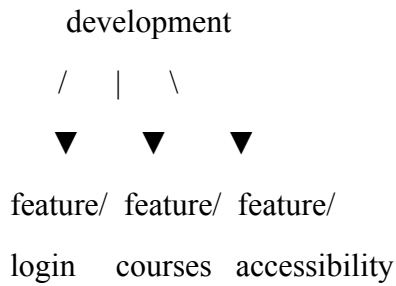
4. Git Branching Strategy

For this project, a simple Main + Development + Feature Branch strategy can be used.

main

|





Branches

Main

Contains the stable version of the project.

Development

Contains integrated development work.

Feature branches

Used for individual features.

Examples:

```
git checkout -b feature/login
```

```
git checkout -b feature/course-management
```

```
git checkout -b feature/voice-navigation
```

```
git checkout -b feature/assessment
```

This prevents different developers from directly changing the stable version.

5. Version Control Workflow

Step 1 – Create Feature Branch

```
git checkout development
```

```
git checkout -b feature/voice-navigation
```

Step 2 – Develop Feature

Implement voice navigation in the application.

Step 3 – Check Changes

```
git status
```

Step 4 – Add Files

```
git add .
```

Step 5 – Commit

```
git commit -m "Add voice navigation feature"
```

Step 6 – Merge

```
git checkout development
```

```
git merge feature/voice-navigation
```

Step 7 – Push

```
git push origin development
```

Step 8 – Final Release

```
git checkout main
```

```
git merge development
```

```
git push origin main
```

6. Commit History Analysis

Example commit history:

Initial project setup

Add Flask application

Add student registration and login

Add course management

Add accessible dashboard

Add voice navigation

Add audio learning resources

Add accessible assessment module

Add progress tracking

Add Docker configuration

Fix accessibility navigation issue

Update project documentation

Good commit messages clearly describe what was changed.

Example:

```
git commit -m "Add accessible student dashboard"
```

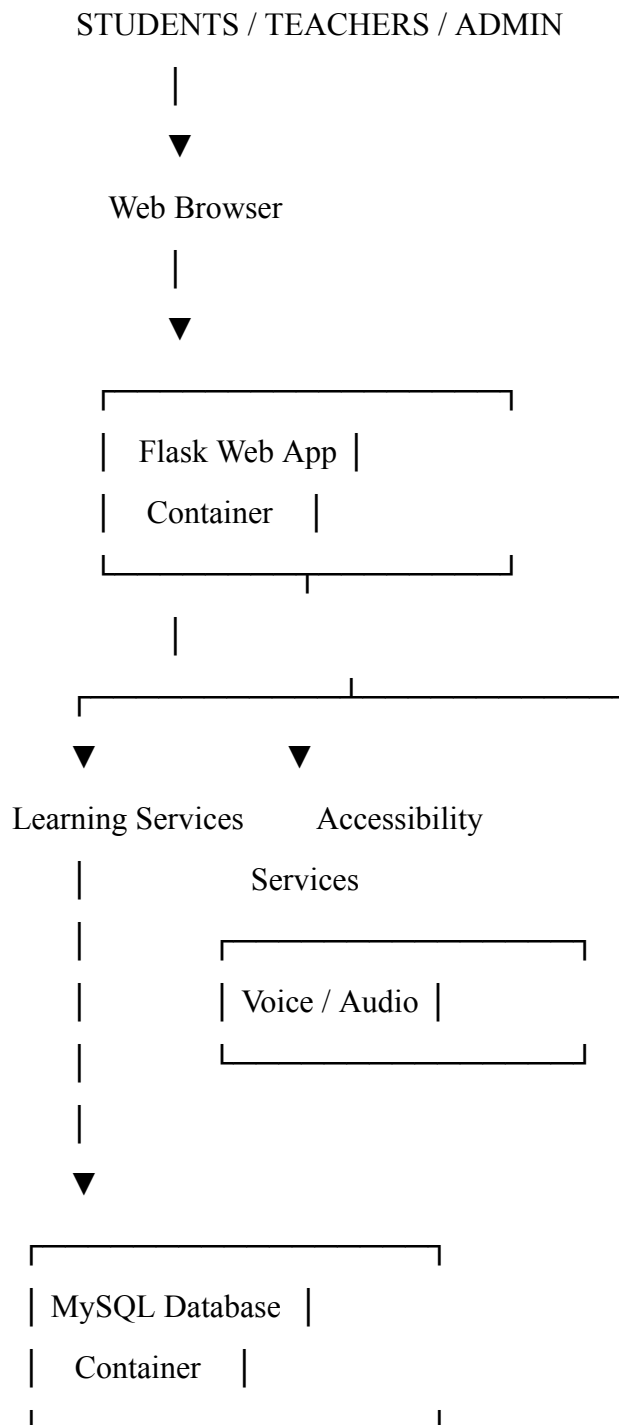
is better than:

```
git commit -m "changes"
```

7. Docker Environment Design

Docker is used to package the Accessible Learning Platform and its dependencies into containers.

Proposed Docker Architecture



Why Docker?

- Provides a consistent development environment.

- Simplifies installation.
- Makes deployment easier.
- Separates application and database.
- Improves portability.
- Helps team members run the same application.
- Makes future deployment easier.

8. Dockerfile Development

Create a file named:

Dockerfile

Dockerfile

FROM python:3.13-slim

WORKDIR /app

COPY requirements.txt .

RUN pip install --no-cache-dir -r requirements.txt

COPY . .

EXPOSE 5000

CMD ["python", "app.py"]

Explanation

Instruction	Purpose
FROM	Selects Python base image
WORKDIR	Sets application directory
COPY	Copies project files
RUN	Installs dependencies

EXPOSE	Opens Flask port
CMD	Starts Flask application

9. Docker Compose Design

Create:

docker-compose.yml

Example

services:

web:

build: .

container_name: accessible_learning_app

ports:

- "5000:5000"

depends_on:

- db

volumes:

- ./uploads:/app/uploads

db:

image: mysql:8.0

container_name: accessible_learning_db

environment:

MYSQL_ROOT_PASSWORD: root

MYSQL_DATABASE: learning_platform

MYSQL_USER: student

MYSQL_PASSWORD: student123

ports:

- "3306:3306"

Services

Web Service

Runs the Flask Accessible Learning Platform.

Database Service

Runs MySQL and stores:

- Student details
- Teacher details
- Courses
- Learning materials
- Assessments
- Results
- Progress

Volume

`./uploads:/app/uploads`

stores uploaded learning resources outside the container.

10. Container Deployment and Testing

Build the application:

`docker compose build`

Start the containers:

`docker compose up`

Or run in background:

`docker compose up -d`

Check running containers:

`docker ps`

Open the application:

`http://127.0.0.1:5000`

Testing

Test the following:

Test	Expected Result
Student Login	Dashboard opens

Teacher Login	Teacher dashboard opens
Course Access	Course content displayed
Audio Content	Audio plays
Voice Navigation	Commands navigate pages
Assessment	Questions are accessible
Submit Assessment	Result is stored
Progress	Student progress displayed
Admin Dashboard	Admin statistics displayed

11. Benefits of Git and Docker

Git Benefits

- Version control
- Easy collaboration
- Branch management
- Change tracking
- Recovery of previous versions
- Better project maintenance

Docker Benefits

- Easy deployment
- Consistent environment
- Portability
- Dependency management
- Application isolation
- Easy testing

Combined Benefit

Git manages the source code, while Docker manages the application environment.

Git

↓

Source Code Version Control

↓

Docker

↓

Containerized Application

↓

Deployment

12. Challenges and Improvements

Challenges

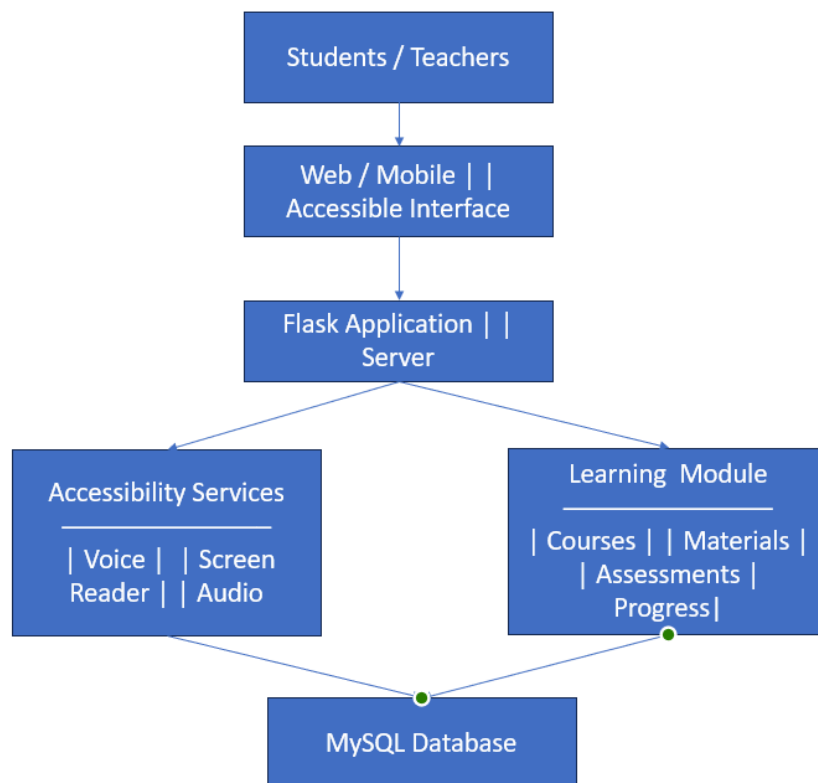
- Making every page screen-reader compatible.
- Implementing reliable voice navigation.
- Supporting different accessibility requirements.
- Managing uploaded audio and document files.
- Connecting Flask with MySQL.
- Configuring Docker networking.
- Maintaining accessibility during UI updates.

Future Improvements

- Add multilingual voice support.
- Add more keyboard navigation.
- Support additional assistive technologies.
- Add mobile application support.
- Add offline learning.
- Add accessibility testing tools.
- Deploy using cloud infrastructure.
- Add real-time teacher-student communication.

13. System Architecture

For your Accessible Learning Platform, use this architecture:



14. Git + Docker Overall Workflow

Developer

|



Source Code

|



Git Repository

|

| — Feature Branch

|

| — Commit

|

| — Merge

|



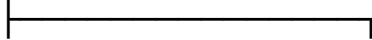
Dockerfile



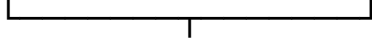
Docker Image



Docker Compose



Flask Container MySQL Container



Accessible Learning

Platform

15. Architecture Justification

The selected architecture uses a three-layer structure:

1. Presentation Layer

Provides the accessible web/mobile interface for students, teachers, and administrators.

2. Application Layer

Flask handles:

- Authentication
- Courses
- Assessments
- Voice services
- Progress tracking
- Reports

3. Database Layer

MySQL stores all application data.

This architecture provides maintainability, scalability, portability, security, and accessibility.

16. Quality Attributes

Quality Attribute	Implementation
Accessibility	Screen-reader and keyboard support
Usability	Simple interface
Performance	Lightweight Flask backend
Security	Login and authentication
Scalability	Docker-based deployment
Maintainability	Modular project structure
Portability	Docker containers
Reliability	Database-backed storage

17. Expected Deliverables

For your assignment, submit:

- Technical report
- Git repository
- Complete source code
- requirements.txt
- Dockerfile
- docker-compose.yml
- Database file/script
- Git screenshots
- Docker screenshots
- System architecture diagram
- Git workflow diagram
- Docker architecture diagram
- Testing screenshots

- Results and conclusion

This structure directly matches the 12 assignment questions and the rubric, while using your Accessible Learning Platform as the project topic.